

Algorithmique 1

2. Grand O

Johan Thapper
2023/2024



Exemple : doublons

- ▶ Considérons la fonction **duplicates** qui cherche des doublons dans un tableau
- ▶ Elle fait $T_{\text{dup}}(n) = n(n-1)/2$ **comparaisons** dans le pire des cas
- ▶ Et si on compte d'autres opérations ?
 - ❖ accès aux éléments du tableau
 - ❖ opérations arithmétiques
 - ❖ conditions
 - ❖ ...
- ▶ Dans ces cas, on obtient peut-être une expression de type $T(n) = 3n^2 + 17n + 5$ (constantes tirées par les cheveux !)
- ▶ Quelle est "la bonne" complexité de **duplicates**, $T_{\text{dup}}(n)$ ou $T(n)$?

```
int duplicates(int t[], int n) {  
    int i, j;  
    for (i = 0; i < n; i++) {  
        for (j = i+1; j < n; j++)  
            if (t[i] == t[j])  
                return 1;  
    }  
    return 0;  
}
```

Exemple : doublons

- ▶ *Notre objectif est de savoir comment croissent les besoins en temps lorsque la taille des entrées augmente.*
- ▶ Quelle est la bonne complexité ?
 - ❖ $T_{\text{dup}}(n) = n(n-1)/2$
 - ❖ $T(n) = 3n^2 + 17n + 5$
- ▶ *Si on double la taille des entrées, alors comment ces fonctions évoluent ?*
 - ❖ $T_{\text{dup}}(2n) = 2n(2n-1)/2 = 2n(n-1/2) \approx 2n(n-1) = 4 T_{\text{dup}}(n)$
 - ❖ $T(2n) = 3(2n)^2 + 17(2n) + 5 = 12n^2 + 34n + 5 \approx 4 T(n)$
(parce que $n^2 \gg n$ si n est grand)
- ▶ Pour les deux fonctions, une augmentation d'un facteur 2 en taille des entrées entraîne (approximativement) un facteur 4 en temps
- ▶ Donc $T_{\text{dup}}(n)$ et $T(n)$ sont toutes les deux "la bonne"

Exemple : doublons

- ▶ Soit $T_{\text{dup}}(n) = n(n-1)/2$ et $T(n) = 3n^2 + 17n + 5$
- ▶ Pour des grandes données, $T(n)$ se comporte comme $T_{\text{dup}}(n)$
 - ❖ à une constante — ici 6 — près

n	$T(n)$	$6 \times T_{\text{dup}}(n)$	$6 \times T_{\text{dup}}(n) / T(n)$
10^1	475	270	0.568
10^2	31705	29700	0.936
10^3	3017005	2997000	0.993
10^4	300170005	299970000	0.999

Simplification

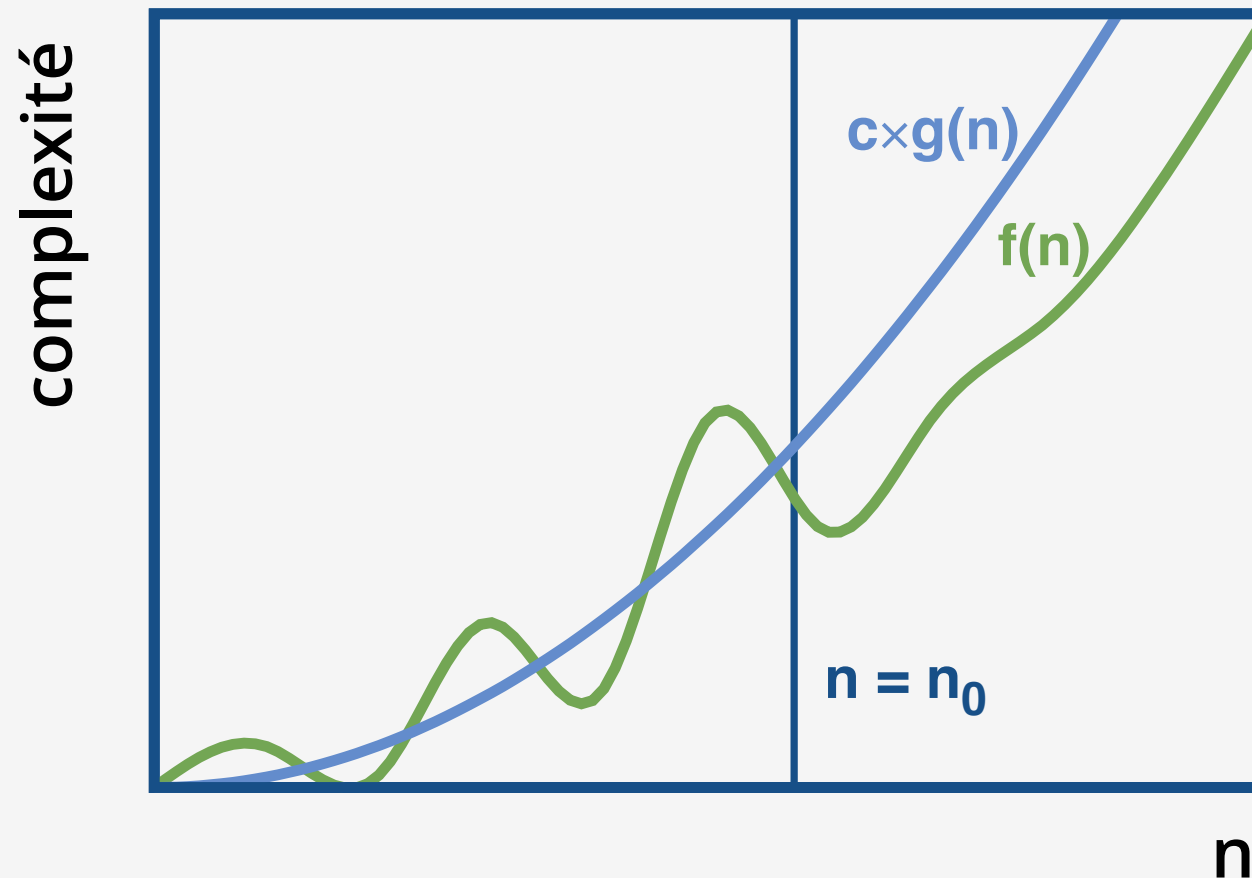
- ▶ Si la complexité $T(n)$ d'un algorithme pour des grandes données se comporte comme n^2 , $T(2n) \approx 4 T(n)$, alors on va simplement la décrire par la fonction $T'(n) = n^2$
- ▶ La complexité de l'algorithme **duplicates** est dominée par n^2
 - ❖ au pire des cas,
 - ❖ pour des grands n et
 - ❖ à une constante près.

Notation grand O

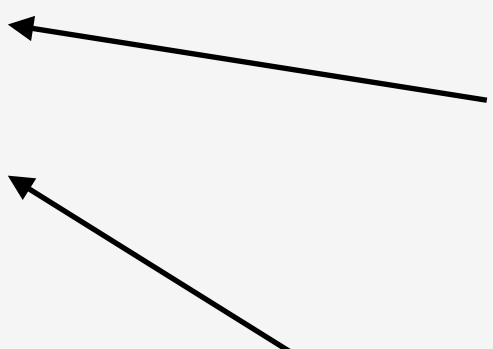
f est (asymptotiquement) **bornée par g** (à une constante près)
s'il existe deux constantes C et $n_0 \geq 0$ t.q., pour tout $n \geq n_0$,

$$f(n) \leq C \times g(n)$$

On note $f(n) = O(g(n))$. On lit "**f est grand O de g**".



Exemples : grand O

- ▶ $3n^2 + 17n + 5 = O(n^2)$
 - ▶ $n^2 = O(n^2)$
 - ▶ $1 + 2 + \dots + n = O(n^2)$ car $1 + 2 + \dots + n \leq n + n + \dots + n = 1 \times n^2$
 - ▶ $n = O(n^2)$
 - ▶ $42 = O(1)$
- 
- Attention !

$T(n) = O(1)$ veut dire qu'il existe deux constantes C et n_0 t.q.

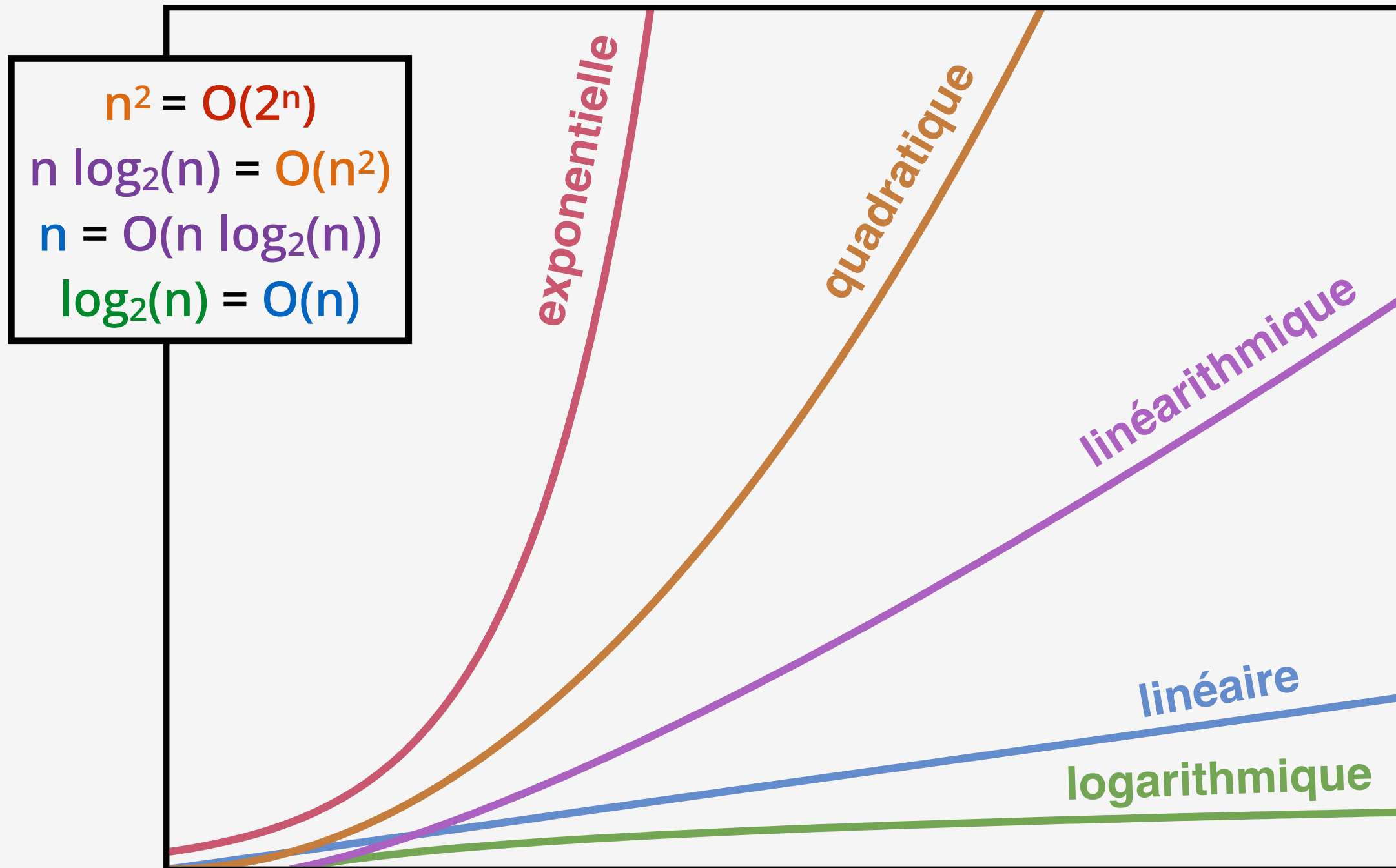
$$T(n) \leq C \text{ pour tout } n \geq n_0$$

" $T(n)$ est finalement bornée par une constante"

Ordres de grandeur

expression	en mots	algorithme	exemple
$O(1)$	constante	décider si un nombre est pair ou impair	décider si on prend la voiture pendant la circulation alternée
$O(\log_2(n))$	logarithmique	recherche par dichotomie	trouver un mot dans un dictionnaire
$O(n)$	linéaire	simple parcours d'un tableau	lire un livre
$O(n \log_2(n))$	"n log n"	tri fusion	classer un jeu de cartes <i>(normalement par un autre tri en temps quadratique)</i>
$O(n^2)$	quadratique	double parcours de tableau	vérifier si le contenu de la liste de courses est dans le chariot
$O(2^n)$ ou $O(c^n), c > 1$	exponentielle	recherche exhaustive (brute force)	essayer tous les mots de passe de longueur n, un par un

Ordres de grandeur



Les règles du jeu

- ▶ Quelques règles utiles

- ❖ $C \times f(n) = O(f(n))$ pour une constant $C > 0$ quelconque

- ❖ “le grand O mange les constantes”

- ❖ $O(f(n)) + O(g(n)) = O(h(n))$ si $f(n) = O(h(n))$ et $g(n) = O(h(n))$

- ❖ $O(f(n)) \times O(g(n)) = O(f(n) \times g(n))$

- ❖ $f(n) = O(h(n))$ si $f(n) = O(g(n))$ et $g(n) = O(h(n))$

- ▶ Exemples

- ❖ $10\log_2(n) = O(\log_2(n))$

- ❖ $10n + O(\log_2(n)) = O(n)$

- ❖ $O(n) \times 10\log_2(n) = O(n \log_2(n))$

- ❖ $n^3 + 5n^2 - 10n + 2^{42} = O(n^3)$

Exemple : la complexité d'une fonction

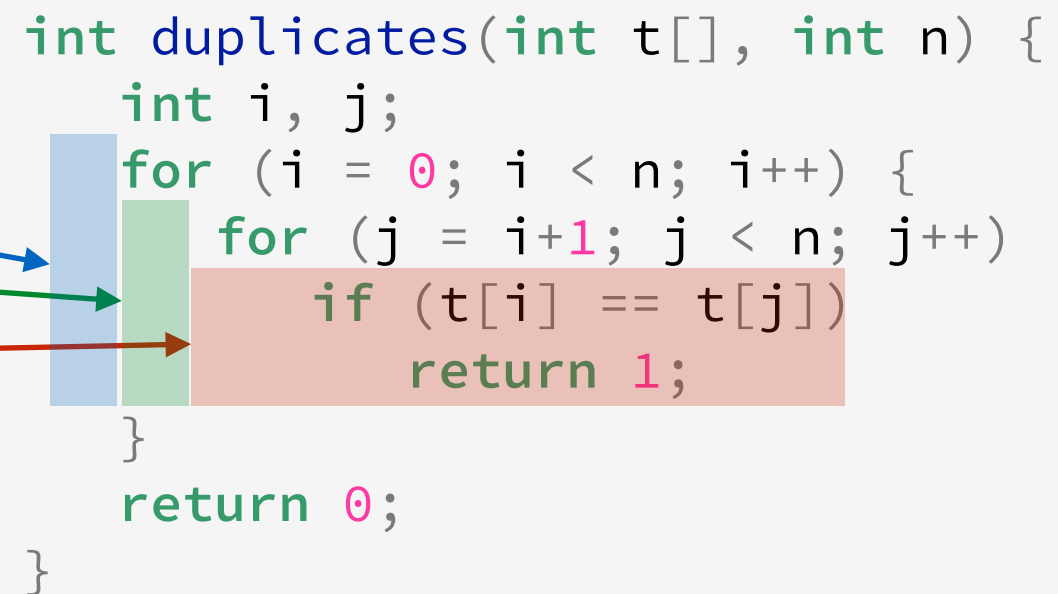
$T(n)$ = nombre de comparaisons

$T(n) = n \times O(n) = O(n^2)$ comparaisons

$O(n) \times O(1) = O(n)$ comparaisons

$O(1)$ comparaisons

```
int duplicates(int t[], int n) {  
    int i, j;  
    for (i = 0; i < n; i++) {  
        for (j = i+1; j < n; j++)  
            if (t[i] == t[j])  
                return 1;  
    }  
    return 0;  
}
```



► Quelques règles utiles

❖ $C \times f(n) = O(f(n))$ pour une constant $C > 0$ quelconque

❖ “le grand O mange les constantes”

❖ $O(f(n)) + O(g(n)) = O(h(n))$ si $f(n) = O(h(n))$ et $g(n) = O(h(n))$

❖ $O(f(n)) \times O(g(n)) = O(f(n) \times g(n))$

❖ $f(n) = O(h(n))$ si $f(n) = O(g(n))$ et $g(n) = O(h(n))$

La complexité du code

- ▶ **Instructions isolées**

- ❖ complexité $O(1)$

Supposer que le code F est de complexité $O(f(n))$ et le code G est de complexité $O(g(n))$

- ▶ **Séquence d'instructions F; G;**

- ❖ complexité $O(f(n)+g(n))$

- ▶ **Conditionnelle** if ($x > 0$) { F } else { G }

- ❖ complexité $O(\max(f(n),g(n))) = O(f(n)+g(n))$

- ▶ **Boucle** for ($i = 0; i < n; i++$) { F }

- ❖ complexité $O(n \times f(n))$

Boucles imbriquées

- ▶ Deux boucles **identiques**
 - ❖ $O(n^2)$
- ▶ Trois boucles identiques
 - ❖ $O(n^3)$
- ▶ Quatre boucles identiques ...
- ▶ Deux boucles **indépendantes**
 - ❖ $O(m \times n)$
- ▶ Deux boucles **dépendantes**
 - ❖ $O(n^2)$
(exemple : duplicates)

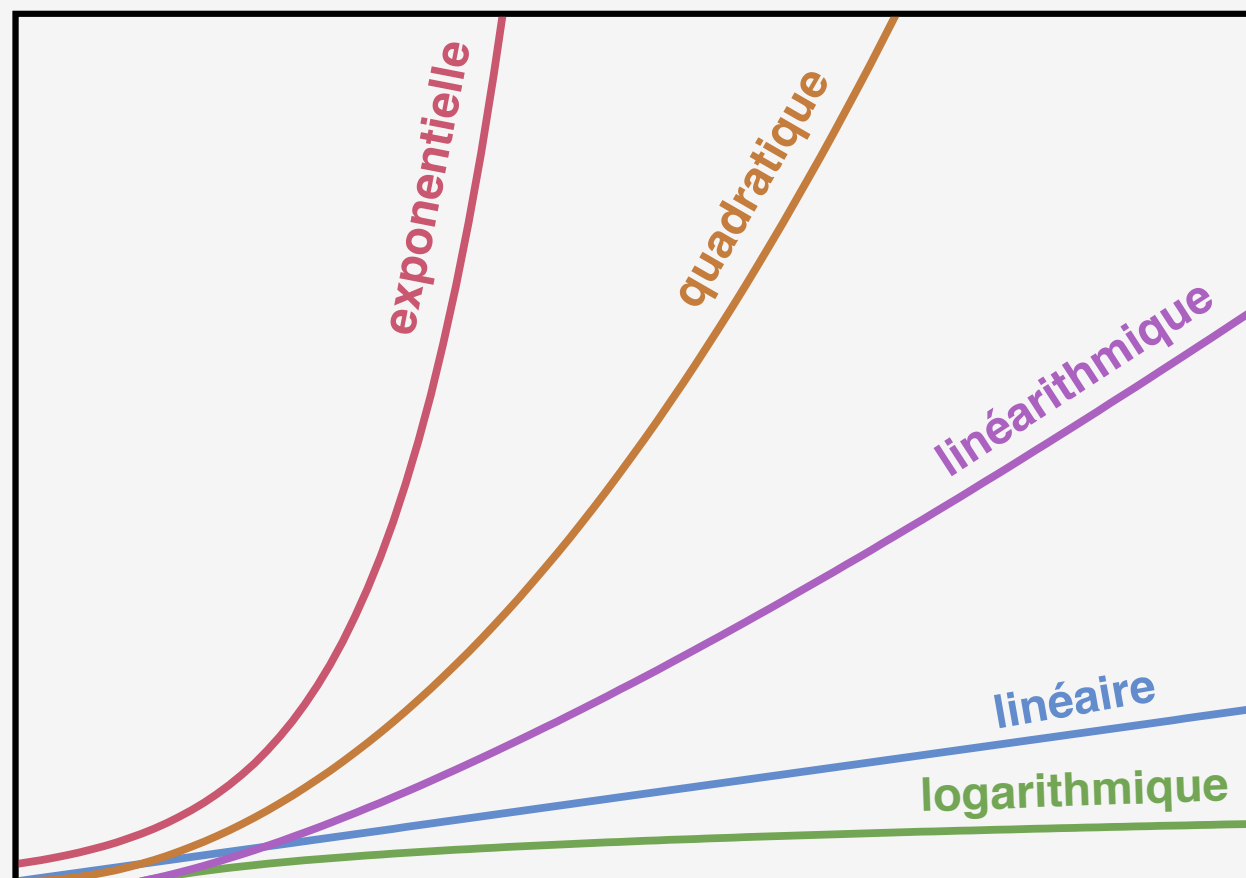
```
for (i = 0; i < n; i++) {  
    for (j = 0; j < n; j++) {  
        /* corps */  
    }  
}
```

```
for (i = 0; i < m; i++) {  
    for (j = 0; j < n; j++) {  
        /* corps */  
    }  
}
```

```
for (i = 0; i < n; i++) {  
    for (j = i + 1; j < n; j++) {  
        /* corps */  
    }  
}
```

Conclusions

- ▶ Pour éviter de calculer en détails la complexité d'un algorithme, on n'étudie pas les résultats précis mais la manière avec laquelle ces résultats évoluent par rapport à la taille de l'entrée
- ▶ Notre objectif est plutôt de savoir **comment croissent les besoins en temps** lorsque la taille des entrées augmente — **son comportement asymptotique**



A retenir

- ▶ On fait une analyse dans le pire des cas et en fonction de la taille des données (n)
- ▶ Quoi compter ?
 - ❖ le nombre de comparaisons d'éléments d'un tableau
 - ❖ le nombre d'affectation / accès à un tableau (accès mémoire)
 - ❖ le nombre d'opérations arithmétiques
 - ❖ le nombre d'itérations de la boucle la plus imbriquée
- ▶ Comment compter ?
 - ❖ argument simple — n comparaisons pour une recherche linéaire
 - ❖ évaluer une série — par exemple, $1+2+\dots+n = n(n+1)/2$
 - ❖ argument particulier — $1+\log_2(n)$ comparaisons pour une recherche par dichotomie
 - ❖ évaluer une récurrence
- ▶ Simplifier avec grand O — $O(n)$, $O(n^2)$, $O(\log_2(n))$

Limitations du pire des cas et grand O

- ▶ Si le nombre d'accès à la mémoire d'un algorithme est de $O(f(n))$, alors le nombre d'accès est **dominé par** $f(n)$:
 - ❖ au pire des cas,
 - ❖ pour des grands n et
 - ❖ à une constante près.
- ▶ Dans **le meilleur cas** (ou **en moyenne**), l'algorithme peut utiliser beaucoup moins d'accès
- ▶ Pour des "**petites**" n , l'algorithme peut se comporter différemment
- ▶ **Les constantes** (n_0 et C) peuvent être **grandes**
- ▶ L'expression $O(f(n))$ cache ces possibilités !